

# Processing Petrological Data with petropandas

## Block 3 — Modern Workflow in Geology

---

Ondrej Lexa

Institute of Petrology and Structural Geology (IPSG)  
Charles University, Prague, Czechia

8 October 2026

# Motivation

---

### Objective

Turn a raw microprobe (EPMA) export into mineral formulas, end-members, a clean set of analyses and publication-ready charts — in one notebook.

1. From Excel to petropandas
2. Units: oxides  $\rightarrow$  moles  $\rightarrow$  cations
3. Crystallochemical formulas & end-members
4. Data cleaning: totals and stoichiometry
5. Charts & export

Notebook: `notebooks/03_petropandas.ipynb`

## The usual Excel workflow

- Copy-paste probe data into a recalculation sheet
- One sheet per mineral, formulas hidden in cells
- Delete “bad” rows by hand — no record of why
- Grt\_recalc\_v3\_FINAL2.xlsx

## A notebook pipeline

- Raw export is **never edited**
- Every step (basis,  $\text{Fe}^{3+}$ , filter) is readable code
- Rejection criteria are explicit and re-runnable
- 10 or 10 000 analyses — same three lines

# What is petropandas?

## In one sentence

A **pandas-accessor library**: oxide wt% → moles → APFU → site-allocated formula → end-members, plus QC, bulk-rock helpers and plotting.

| Accessor              | Purpose                                                 |
|-----------------------|---------------------------------------------------------|
| df.oxides             | recognise & clean oxide wt% columns, filter rows        |
| df.moles / df.cations | wt% ↔ moles ↔ APFU                                      |
| df.mineral            | structural formulas, end-members, stoichiometry QC      |
| df.bulk               | whole-rock: CIPW norm, ratios, THERMOCALC/MAGEMin input |
| plotting              | ScatterPlot, TernaryPlot, ProfilePlot                   |

No new file format, no new data structure — just a DataFrame.

## **From Excel to petropandas**

---

# Loading an Instrument Export

```
from petropandas import pd, Grt, Fsp, Bt, Ms

df = pd.read_excel("data/epma_export.xlsx",
                  sheet_name="Spots")
df.oxides() # only oxide columns

# mineral is hidden in the comment: S12_grt_03
df["Mineral"] = (df["Comment"]
                .str.split("_").str[1])
grt = df.oxides.select("grt", on="Mineral")
```

- Plain `pd.read_excel / read_csv` — no special reader
- Column names parsed as formulas; Point, Comment, Total ignored
- Vendor variants mapped automatically: FeO\*, FeOT, FeO tot → FeO; H2O+ → H2O
- `.select()` takes a substring, a list or a boolean mask

# Units: Oxides → Moles → Cations

```
grt.oxides() # wt%  
grt.moles() # molar proportions  
apfu = grt.cations(n_oxygens=12) # atoms per formula unit, 12 O basis  
apfu.attrs["petro_units"] # 'apfu' - unit is tracked  
apfu.oxides() # ... and conversions are reversible  
  
apfu.cations.calc({"XMg": "Mg{2+} / (Mg{2+} + Fe{2+})",  
                  "Xsite": "Fe{2+} + Mg{2+} + Ca{2+} + Mn{2+}"})
```

- Ions are named with their charge: Si{4+}, Fe{2+}, Na{+}
- `df.cations(n_cations=5)` for a cation-normalised basis (e.g. feldspar)

# Crystallochemical Formulas

---

# Mineral Formulas and End-Members

```
grt.mineral.apfu(Grt) # APFU incl. Fe3+/Fe2+ split (Droop 1987)  
grt.mineral.site_allocations(Grt) # structural formula: (site, cation) columns  
grt.mineral.end_members(Grt) # Prp, Alm, Sps, Grs, Adr, Uvr (%)  
  
pl.mineral.end_members(Fsp) # An, Ab, Or  
bt.mineral.end_members(Bt) # Phl, Ann, Eas, Sid, Dioc
```

## The pattern

Same three calls for every mineral — only the **mineral object** changes. It carries the oxygen basis, ideal cation sum,  $\text{Fe}^{3+}$  method, site scheme and end-member set.

## 16 Built-in Mineral Definitions

| Mineral             | Object       | O basis    | Fe <sup>3+</sup>     | End-members                              |
|---------------------|--------------|------------|----------------------|------------------------------------------|
| Garnet              | Grt          | 12         | Droop                | Prp, Alm, Sps, Grs, ...                  |
| Feldspar            | Fsp          | 8          | —                    | An, Ab, Or                               |
| Clinopyroxene       | Cpx          | 6          | Droop                | Jd, Ae, Wo, En, Fs                       |
| Orthopyroxene       | Opx          | 6          | Droop                | MgTs, Wo, En, Fs                         |
| Muscovite / Biotite | Ms / Bt      | 11         | —                    | Ms, Pg, Cel, ... / Phl, Ann, Sid,<br>... |
| Amphibole           | Amp          | 23         | Schumacher           | 13 end-members (Tr, Prg, Glau,<br>...)   |
| Chlorite            | Chl          | 14*        | —                    | Clin, Cham, Pnn, Sud, ...                |
| Epidote / Titanite  | Ep / Ttn     | 12.5 / 5   | all Fe <sup>3+</sup> | Czo, Ep, ... / Ttn, Al-Ttn, ...          |
| St, Cld, Crd        | St, Cld, Crd | 48, 12, 18 |                      |                                          |
| Ilmenite, Spinel    | Ilm, Spl     | 3, 4       | Droop                |                                          |

\*28-charge normalisation. `from petropandas import mdb; mdb.all()` lists them all; `GrtFe3` estimates Fe<sup>3+</sup> by matrix inversion. THERMOCALC a–x models are in `petropandas.hpxeos`.

# Data Cleaning

---

# Two Independent Quality Checks

## 1. Analytical total

≈ 98.5–101.5 wt% anhydrous,

≈ 94–97 wt% micas

## 2. Stoichiometry

check\_stoichiometry(M) scores each  
criterion 0–1: analytical\_total,  
cation\_deviation, charge\_balance,  
site\_vacancies, leftover\_cations, ...  
stoichiometry\_quality(M) = one number

```
def clean(data, mineral,
          total=(98.5, 101.5),
          min_quality=0.97):
    tot = data.oxides().sum(axis=1)
    q = data.mineral\
        .stoichiometry_quality(mineral)
    ok = (tot.between(*total)
          & (q >= min_quality))
    return data.oxides.select(ok)

grt_ok = clean(grt, Grt)
bt_ok = clean(bt, Bt, total=(94, 97.5))
```

# Why Both Checks?

## Low total, perfect formula

A spot on a crack gives a total of  $\sim 88$  wt% — but all oxides are reduced *proportionally*, so the recalculated formula is **perfect**. Only the total catches it.

## Good total, wrong formula

A garnet spot partly on a plagioclase inclusion has a perfect total ( $\sim 99.7$  wt%), but the cation sum and site occupancy are off. Only stoichiometry catches it.

## Rule

Combine the two, keep the thresholds in code, and **print what was rejected** — your filter becomes part of the methods section.

## Charts & Export

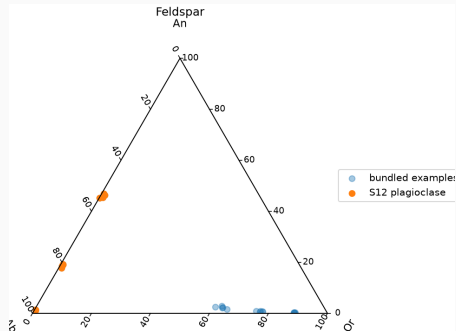
---

# Three Plot Classes, One Pattern

```
from petropandas import (TernaryPlot,
                          ScatterPlot, ProfilePlot)

t = TernaryPlot("An", "Ab", "Or")
t.add(pl_ok.mineral.end_members(Fsp),
      label="S12 plagioclase")
t.show()

s = ScatterPlot("Si{4+}",
               "Fe{2+} / (Fe{2+} + Mg{2+})")
s.add(bt_ok.mineral.apfu(Bt), label="Bt")
s.savefig("output/micas.pdf")
```

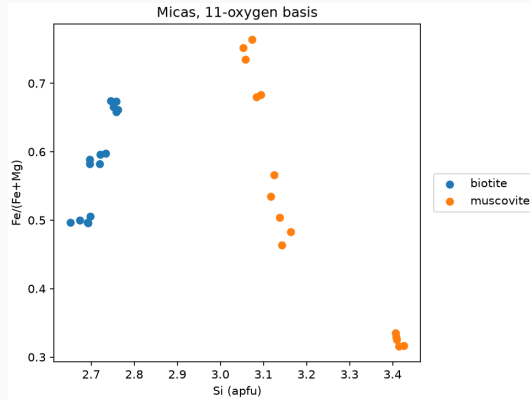


Axes are **expressions** — no helper columns needed.

# Examples from the Notebook



ProfilePlot: garnet traverse, split="auto"



ScatterPlot: Si vs. Fe/(Fe+Mg) in micas

# Exporting Results

```
with pd.ExcelWriter("output/S12_results.xlsx") as xls:
    grt_ok.mineral.apfu(Grt).round(4).to_excel(xls, sheet_name="Grt apfu")
    grt_ok.mineral.end_members(Grt).round(2).to_excel(xls, sheet_name="Grt end-members")
    pl_ok.mineral.end_members(Fsp).round(2).to_excel(xls, sheet_name="Pl end-members")

p.savefig("output/grt_profile.pdf") # vector graphics: .pdf / .svg
```

- Results are ordinary DataFrames — any pandas writer works (to\_excel, to\_csv, to\_latex)
- Whole-rock data? df.bulk.cipw(), .oxide\_ratios(), .TCbulk(), .MAGEMin()

## Hands-on

---

### **Muscovite analyses in `data/epma_export.xlsx` (notebook section 3.7)**

1. Select the `ms` rows and recalculate them with `Ms`
2. Clean them with `clean()` — choose a sensible total window for a hydrous mineral
3. End-members and Si (apfu): is there a phengite (celadonite) component?
4. ScatterPlot of Si vs.  $\text{Na}/(\text{Na}+\text{K})$ , saved as SVG to `output/`
5. *Bonus*: mean matrix garnet vs. rim of the garnet profile

## Recap

---

### Key takeaways

- Raw export → APFU → end-members, all on a DataFrame
- One mineral object carries the whole recalculation recipe
- Filter on **total and stoichiometry**, and log what you reject
- Expression-based plots, vector output, Excel export

**Resources:** [github.com/ondrolexa/petropandas](https://github.com/ondrolexa/petropandas) (docs & walkthrough notebook)

Next: Bring Your Own Data — `04b_byod_epma.ipynb`